

TEMA 3 - FUNCIONES EN PYTHON

Francisco Gortázar



DEFINICIÓN

- Sintaxis

```
1 def my_function(arg1, arg2, arg3):  
2     # Código de la función
```

- `arg1`, `arg2` y `arg3` son **parámetros** de la función



DEFINICIÓN

- Ejemplo:

```
1 def fibonacci(num_terminos):  
2     a = 0  
3     b = 1  
4     while num_terminos > 0:  
5         print(a)  
6         a = b  
7         b = a + b  
8         num_terminos -= 1
```



LLAMADO

- Sintaxis

```
1 my_function(arg1, arg2, arg3)
```

- `arg1`, `arg2` y `arg3` son **argumentos** posicionales de la función
 - A qué parámetro se asigna cada uno depende de su posición
- Ejemplo:

```
1 fibonacci(10)
```



RETORNO DE VALORES

- Sintaxis

```
1 def my_function(arg1, arg2):  
2     return arg1 + arg2
```

- Ejemplo

```
1 def fibonacci(num_terminos):  
2     a = 0  
3     b = 1  
4     while num_terminos > 0:  
5         a = b  
6         b = a + b  
7         num_terminos -= 1  
8     return a
```

- Llamada

```
1 value = fibonacci(10)
```



RETORNO DE VALORES

- Podemos devolver múltiples valores:

```
1 def minmax(a, b):  
2     if a < b:  
3         return a, b  
4     else:  
5         return b, a
```

- Uso:

```
1 min, max = minmax(20, 10)  
2 print(min, max)
```



ARGUMENTOS NOMBRADOS

- Se pueden pasar argumentos a una función indicando el nombre del parámetro al que se asignan
- Flexibilidad: el orden de los argumentos no importa
- Claridad: permite saber exactamente qué valor corresponde a cada parámetro en la definición

```
1 value = fibonacci(num_terminos=10)
2 print(value)
```

- `num_terminos` es un argumento nombrado: se especifica a qué parámetro de la función se asigna el valor `10`



ARGUMENTOS NOMBRADOS

- Son mucho más claros en contextos donde no sabemos en qué orden suministrar los argumentos:

```
1 generate(True, False)
```

- ¿Qué hace el primer `True`? ¿Y el segundo `False`?
- Usando argumentos nombrados se clarifica el código:

```
1 generate(createFileIfNotExists=True, overwrite=False)
```



PARÁMETROS POR DEFECTO

- Es posible asignar un valor predeterminado a un parámetro
- Los parámetros con valores por defecto deben ir al final de la lista de argumentos de la definición de la función para que los argumentos posicionales sigan funcionando
- Si al llamar a la función se omite el argumento, se usa el valor por defecto



PARÁMETROS POR DEFECTO

- Ejemplo:

```
1 def fibonacci(num_terminos=10):  
2     a = 0  
3     b = 1  
4     while num_terminos > 0:  
5         a = b  
6         b = a + b  
7         num_terminos -= 1  
8     return a
```

- El valor de `num_terminos` será 10:

```
1 value = fibonacci()  
2 print(value)
```



PARÁMETROS POR DEFECTO

- Ejemplo incorrecto:

```
1 def fibonacci(num_terminos=10, init_termino):  
2     # Código...
```



PARÁMETROS OPCIONALES

- Para que un argumento sea opcional, se le suele asignar el valor especial `None` por defecto en la definición
- En el cuerpo de la función, se utiliza una estructura condicional (`if`) para comprobar si se proporcionó un valor o si el parámetro permanece como `None`
- Útil cuando se desea que una función maneje información extra solo si el usuario decide enviarla



PARÁMETROS OPCIONALES

- Ejemplo:

```
1 def fibonacci(num_terminos=None):
2     # En realidad se podría hacer con un parámetro por defecto
3     if num_terminos is None:
4         num_terminos = 10
5     a = 0
6     b = 1
7     while num_terminos > 0:
8         a = b
9         b = a + b
10        num_terminos -= 1
11    return a
```

```
1 value = fibonacci()
2 print(value)
```



PASO DE FUNCIONES A FUNCIONES

- Las funciones son ciudadanos de primer orden
 - Pueden pasarse como argumentos
 - Pueden asignarse a variables

```
1 def starts_with_a(w):  
2     return w.startswith("a")  
3  
4 li = ["apple", "banana", "avocado", "cherry", "apricot"]  
5 res = filter(starts_a, li)  
6 print(list(res))
```



FUNCIONES LAMBDA

- Son funciones anónimas y se definen con la palabra clave lambda
- Restricción
 - Están limitadas a contener una única expresión en lugar de múltiples sentencias de código
- Uso
 - Se utilizan frecuentemente en programación funcional para operaciones rápidas que se pueden pasar como objetos a otras funciones

```
1 li = ["apple", "banana", "avocado", "cherry", "apricot"]  
2 res = filter(lambda w: w.startswith("a"), li)
```



```
3 print(list(res))
```

```
1 l = ["apple", "banana", "avocado", "cherry", "apricot"]  
2 l.sort(key=lambda w: len(w))  
3 print(l)
```

